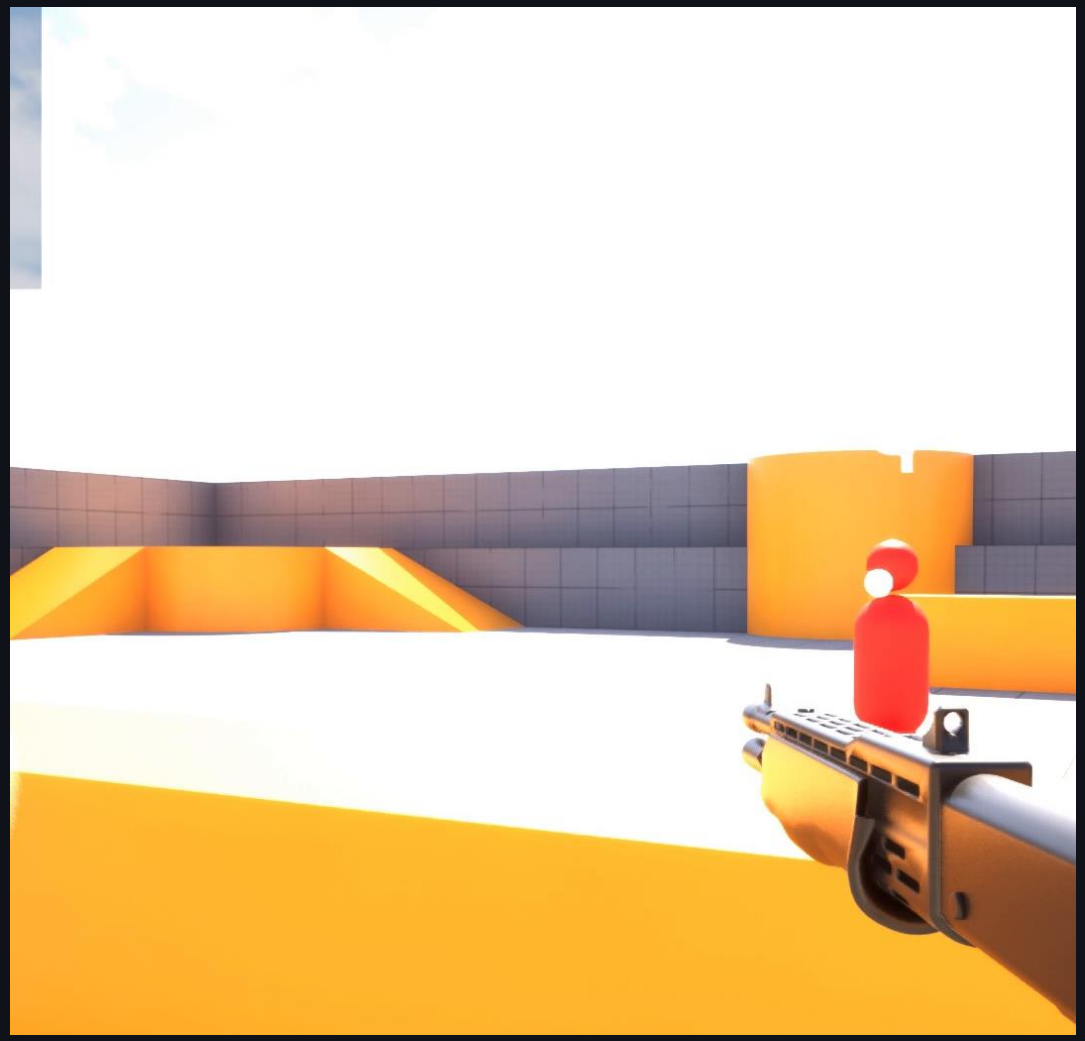
DosCoiDev

Game Experience Orchestration Middleware

External encounter-level Director layer for Unreal Engine prototypes



Observation JSON in · Validated Director Command JSON out · Engine remains in control

Epic MegaGrants pitch draft

Executive summary

A middleware that coordinates encounters—not just enemies.

DosCoiDev adds an external orchestration layer above existing game AI.
Instead of making each enemy more complex, it coordinates roles, timing, pressure, and engagement while Unreal Engine remains in control.

Key Value

- Keeps existing Behavior Trees and local AI intact.
- Coordinates encounters rather than replacing enemy AI.
- Produces validated, bounded Director Commands.
- Supports both RuleSet and LLM Director paths.

Current Proof

- Unreal Engine 5 prototype
- One-versus-many FPS demo
- Baseline -> RuleSet -> LLM comparison
- Observation -> Director -> Validator -> UE Executor

The engine keeps execution authority.

The experience can change. The engine remains in control.

The problem

Local AI behavior is necessary, but encounters are still hard to direct, test, and iterate on.

TODAY

Enemy BT / GOAP handles individual behavior.

Encounter designers still need higher-level control:
where pressure comes from, when to rotate, when to back off.

GAP

Hand-written rules are fast but rigid.

LLMs are flexible but unsafe without contracts.
Game engines need validated, bounded commands.

The missing layer is a safe, external Director interface between high-level intent and engine-owned execution.

What DosCoiDev is

A middleware pattern for encounter-level orchestration, not a replacement for BT, GOAP, or local navigation.

Engine-owned: pawn control, navigation, animation, combat execution.

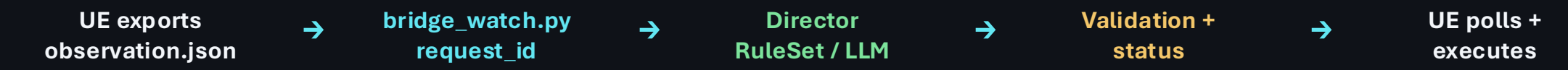
DosCoiDev-owned: interpreting state and selecting bounded encounter intent.

Validator-owned: schema, policy, cooldowns, safety constraints.

Core principle: “The experience can change. The engine remains in control.”

Current prototype architecture

A working file-based loop for controlled UE proof-of-concept demos.



Working now

request_id handshake, bridge_status ready, UE Poll execution loop

Demo commands

flank_pressure_2 and shotgun_fallback mapped to UE behavior

Safety

existing enemies only, no spawn, no reset, no teleport

Measured prototype latency: RuleSet ≈0.2 sec. LLM path ≈8 sec via API + subprocess + file polling. Roadmap focuses on schema freeze first, then lower-latency backend paths.

Demo structure

The pitch video compares three modes with the same arena and existing enemies.

Baseline

Local AI only

RuleSet

Fast fixed route command

LLM Director

Context-aware weapon response

Video narrative: local behavior → deterministic orchestration → richer external Director intent.

Baseline: local AI only

Comparison starts with no external Director active.

WHAT THE VIEWER SHOULD SEE

- Enemies keep frontal pressure.
- No route selection is issued.
- No weapon-threat response exists.
- This is not “broken AI”; it is the baseline without encounter-level orchestration.

RuleSet Director: fast but fixed

A deterministic policy can issue bounded UE commands immediately.

OBSERVED IN UE

Event Type: FLANK

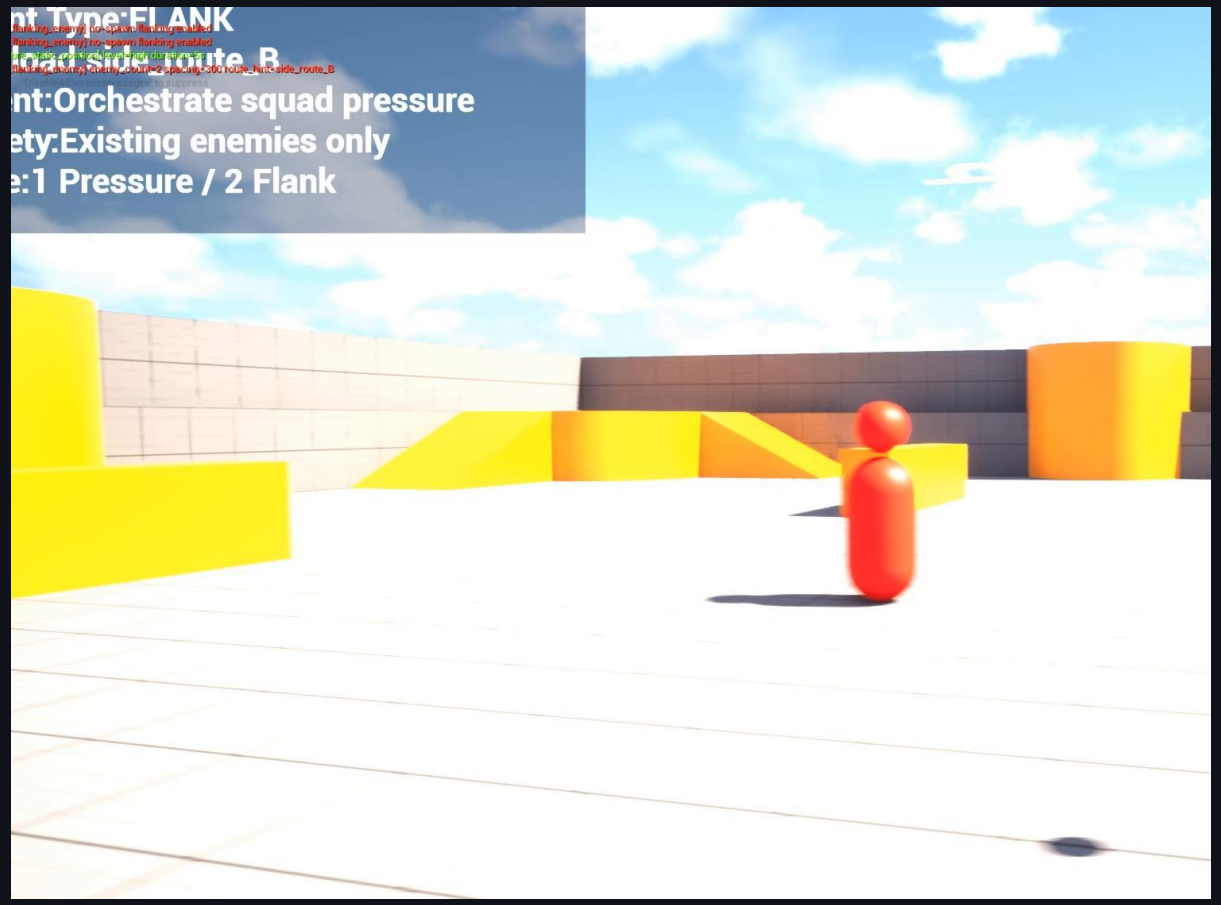
Payload: side_route_B / C / A

Existing enemies execute prepared route bindings.

Strength: measured low latency (≈0.2 sec) and predictable behavior. Limitation: context is limited to authored rules.

LLM Director: richer context, bounded output

The LLM path is not framed as a frame-perfect reflex system.



LLM ROLE

- Receives structured observation summary.
- Chooses one high-level Director intent.
- Output is restricted to schema-valid decisions.
- UE remains responsible for movement and combat.

Measured prototype latency: ≈8 sec. Positioning: This is not a reflex loop; it is intended for encounter-level decisions where richer context matters.

Key moment: shotgun threat response

The strongest demo beat shows the LLM selecting a different encounter intent when the player changes weapon context.

LLM OUTPUT

Event Type: SHOTGUN

Payload: cautious_distance

Intent: Fallback

Role: Fallback

Interpretation: the Director changes squad-level engagement range; it does not teleport or directly control pawns.

Safety and engine control are design features

The prototype intentionally keeps authority inside Unreal Engine.

The Director selects intent.
The engine executes behavior.

The UE side owns navigation, target points, enemy movement, line of sight, and combat execution.

DosCoiDev sends bounded commands only after validation.

HARD DEMO CONSTRAINTS

- Existing enemies only
- No spawn during FEAR demo
- No reset command
- No teleport
- No direct pawn control from LLM

Differentiation: not another internal AI Director

Classic AI Director patterns are usually tightly integrated into a specific game. DosCoiDev explores a reusable external boundary for UE prototypes.

CLASSIC AI DIRECTOR

Game-specific internal pacing / spawn / pressure logic.

Tightly coupled to one title, one ruleset, and one runtime implementation.

Good for shipped designs, but harder to swap between RuleSet / LLM paths.

DOSCOIDEV BOUNDARY

Observation JSON → External Director → Validator → Director Command JSON.
UE keeps execution authority; DosCoiDev makes intent inspectable, swappable, and safety-bounded.

Positioning: DosCoiDev does not claim to invent AI Director design. It tests a safer middleware seam for modern UE workflows.

Roadmap: FPS proof → reusable UE middleware

Current proof stays narrow: one-versus-many FPS. Grant work turns that proof into a reproducible boundary and documented extension path.

MILESTONES

v0.83 Proof Lock: freeze Observation / Command schema; keep verified FPS command families only.

v0.90 Reproducibility: structured logs, latency measurement, route inspection, UE adapter cleanup.

v1.0 UE Proof Build: sample project, docs, demo video, and technical write-up for Unreal developers.

GENERALIZATION PATH

- Player-visible beats first: flank pressure, shotgun fallback, recovery window
- Preset families: Flank Pressure, Range Response, Recovery Window
- Future domains as adapters: horde, squad, RPG, turn-based — not claimed as current proof
- Investigation notes: BT / Blackboard adapters and character AI systems such as NVIDIA ACE

Goal: make the FPS proof reproducible while clearly separating what is validated now from what the middleware boundary can support next.

Honest risks, measured latency, and mitigation

The current proof is promising, but the next step is disciplined engineering and reproducible measurement.

MEASURED PROTOTYPE LATENCY

RuleSet path: ≈0.2 sec

Suitable for fast deterministic presets and authored encounter rules.

LLM path: ≈8 sec

Too slow for frame-perfect reactions; positioned as encounter-level direction, not reflex AI.

KEY RISKS

- LLM backend needs optimization or use in slower decision loops
- Route selection needs better inspection and reproduction
- Command schema must freeze before expansion

MITIGATIONS

- Keep RuleSet path for low-latency presets
- Treat LLM as high-level Director only
- Freeze two UE-safe command families before adding more

Submission discipline: no big rewrites; schema-first development; backup / diff / rollback for every change.